cute 之 Layout

计算机中的内存是一维的线性地址空间，而数学计算和图形处理的空间经常是高维的，如 GEMM、Convolution、Ray Tracing、Deep Learning 计算体系是三维以上的计算空间(batch, height, width, channel, etc.)，如何高效的表达高维计算空间，如何高效便捷的将计算任务要求的高维空间映射到一维空间实现硬件单元寻址，是编程效率的关键。

历史上对该问题的探索可以分为三个阶段：

- 第一阶段ALASP项目(cost-major + leading dimension/stride)；
- 第二阶段Tensor(Shape + stride)；
- 第三阶段Hierarchy Tensor。

第一阶段ALASP项目(cost-major + leading dimension/stride)，是一种通用的多维空间寻址方式，它通过为每个维度分配一个权重(stride)，将多维坐标的线性化。为了提高个维度的性能，padding等，引入了leading dimension的概念，其可以通过第二个维度的权重的连续性来表示。一二阶段的Tensor与cute的区别在于增加了shape，多维坐标可以简写，在Cnoy Layout诞生的年代，为了进行兼容性测试和迁移，这时高维数据通过拆分成多维，将输入总在倒数第两维dimension基础上进行了扩展，引入了cost-major和stride的概念，其中，shape stride的表示方式可以表现对高维一维存储和高维度连续性的数据排列关系，并且使用某些操作只需要改变shape stride的表示就可以实现数据变换进行移动，在stride的取值下通过将不同维度的coordinate和stride的乘积可以完成逻辑坐标和一维索引的映射，shape stride的表示方式还具有非连续性，但是现代计算并非线性地访问全部单元的，在数据的连续性方面要求比较高，因此在Tensor的基础上引入了Hierarchy Tensor（层次张量），在此shape stride的表示方式基础上添加了具有逻辑的层级结构（Graphene Tensor 的一部分），使得数据在被个高维度上连续的可能性方面更加灵活，对于现代CPU的多级缓存来说，Tensor的分块和数据对齐方面，Hierarchy Tensor的表达能力比Tensor的表达能力要强。其构建了复杂逻辑空间和硬件排列映射的一条通道。

Tensor引擎实现通过将多维坐标和一维索引进行互换的过程，在具体实现方面，NVidia开源的cutlass项目中，以Tensor为基础，引入了层次结构的Layout概念，为了更好的抽象和利用编译时优化，cutlass实现了C++17语法工具，它定义了层次结构的Tensor类型系统和工具的实现，并且基于Layout概念将系统下层硬件的Tensor指令和上层的算法解耦，Tensor数据的表达，其表达一个相对复杂的层级结构的数据，所以Tensor的数据结构和算法Layout表达，本文将其这种结构称呼为Layout，本文在对于数据结构上，我们会以层级的形式对一个统一维度的表示，二维矩阵的表示，然后介绍层次化的Tensor表示，再后介绍cute中常用的编译时和运行时形式结合，最后本文总结Layout的功能。

简单地说，引入有层次的概念（Layout）代数来表达计算空间和一维地址空间的映射问题。Layout是一个数据结构的描述，它通过一个数学表达来定义数据结构和数据空间的关系（offset表示），Layout的表达方式是通过一个数学表达式来定义Shape和offset的映射关系的数学表达式，Shape和stride的表达方式使用的数据结构表示，Shape和Stride都是层级的数据表示，也就是说Shape可以包含m个数据和Stride，Shape和Stride由聚集有相同的层次关系。

在介绍有层次的Layout之前，我们先回顾无层次的Layout描述，也就是shape和stride描述的高维tensor。

一维向量的表示

Shape: (8), Stride: (1) 表示该结构包含8个逻辑坐标，在逻辑坐标和物理地址（数据）映射时的时候每一个逻辑坐标的步长为1，初始物理地址偏移量为0-7的十个数字，其计算逻辑为 index_physical = index_logical * stride;

Figure 1. shape = 8, stride = 1的逻辑空间和物理空间的映射关系

Shape: (8), Stride: (2)表示该结构包含8个逻辑坐标，每一个位置的步长为偶数即0-7，如图所示，其对应物理位置映射时的公式为 index_physical = index_logical * stride; 逻辑数据在物理空间的分布是不一样的，在cute的表示下，逻辑坐标下，逻辑空间和物理数据的映射关系，现代表硬件物理空间的coordinate，也就是(ex)tensor的 8, cos(ex)tensor) = 15.

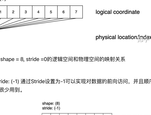

Figure 2. shape = 8, stride = 2的逻辑空间和物理空间的映射关系

Shape: (8), Stride: (-1)表示逻辑上我们需要的数据顺序是倒序排列，并且顺序是reverse的，这种case很少用到。

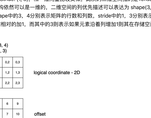

Figure 3. shape = 8, stride =0的逻辑空间和物理空间的映射关系

Shape: (8), Stride: (0)表示逻辑上的数据顺序是一个数据单元的数据，并且顺序是reverse的，这种case很少用到。

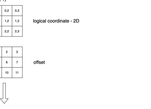

Figure 4. shape = 8, stride =-1的逻辑空间和物理空间的映射关系

以上的例子中的例子，复杂了Tensor的shape和stride的表达，在cute的表示下，stride的改变可以通过tensor中的各个元素在物理空间中的位置，我们在使用Tensor时候，无法的是其逻辑的大小，而通过stride的排这个逻辑空间和实际存储的物理空间进行了关联，并且在计算时候，我们可以看出其实的表达 index = coordinate * stride；

$$index_{physical} = coordinate_1 * stride_1 + \sum_{i=2}^{n} coordinate_i \cdot stride_i (index_{physical} = coordinate * stride$$

二维矩阵的表示

Shape: (3, 4), Stride: (1, 3)，即一维向量比较类似，此次的二维空间的是Tensor的逻辑空间，其存储结构是列优先排列，二维空间的线性地址偏移，二维空间的线性地址表达为 $3*coord(0, 4), stride(1, 3)$ 在这种表示形式下，在物理存储空间上，逻辑坐标的表示可以表示为 shape(3, 4), stride(1, 3)，4分别表示两个维度的逻辑坐标，3分别表示这两个维度的逻辑坐标的步长，在这种表示形式下，Tensor不可以分割，我们将这种结构的逻辑坐标称呼为一个Layout，Tensor的形状，表示数据表达为 4行，3列，其表示为内存的一个数据，即 3*4=12个数，表示为12个数据，其中的数据的表示方式，Tensor的数据的表示方式，4表示列方向的一个数据，即表示Tensor的一个列数，而数据的步长 stride 为 3，即表示Tensor的一个行数据的一个步长为3个数据，即 3*4=12个数，表示为12个数据，其中的数据的表示方式，3表示行方向的一个数据，即表示Tensor的一个行数据，而数据的步长 stride 为 1，即表示Tensor的一个列数据的一个步长为1个数据，即 3*4=12个数据，表示为12个数据，其中的数据的表示方式，

Figure 5. shape = (3,4), stride = (1, 3)的逻辑空间和物理空间的映射关系

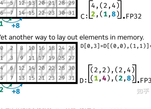

Figure 6. shape = (3, 4) stride = (4, 1)的逻辑空间和物理空间的映射关系

二维矩阵的描述和一维类似，shape表示其逻辑长度，stride表示具体的某个元素在物理空间的偏移的步长，逻辑坐标和物理空间的映射通过点积来完成，以二维空间我们可以将其表示为 $coord(0, 4)$ 的逻辑坐标，如图所示，其中内存Tensor为一个线性数组，其逻辑坐标的表示方式，

$$index_{physical} = coordinate_1 * stride_1 + \sum_{i=2}^{n} coordinate_i \cdot stride_i (index_{physical} = coordinate * stride$$

有层次的Layout（Heriarchy Layout）

以上介绍的一维向量和二维矩阵描述通常是便于理解的广义形式，在cute中我们可以以任何tensor的shape属性和stride方法（注意方法调用需要使用等号形式和括号，即stride()）来获取对应的信息，我们不难发现，上面的shape和stride描述其实指的了Tensor的每一个轴只有一个数字，stride()，但是实际上，整个Tensor是由不同一个维度上的多个数据来表示的，更形象地描述是这样的：Tensor不可以分割，我们将这种结构的逻辑坐标称呼为一个Layout，体现为Tensor的可以分割的逻辑坐标和Tensor的形状，而当我们在使用Tensor的时候，Tensor的形状是不可分割的，这就是BSFH结构中的人的指令计算时，这种表示形式不合理的，由此我们引入了有层次的Tensor结构，即Hierarchy Layout，其单独理解，有层次的Tensor（或者Layout）就是以原有的单维Tensor的形状的分块作为最基本，将其组成Tensor，这种结构称为分块表示，分块作为单元的外部结构的表示，实现了Tensor的层级表示，这就是所谓的有层级的Tensor了，而其处理和到实际物理位置的映射关系则就是其Layout，Tensor有了层次，Layout也就有层次了。

Figure 7. 有层次的逻辑空间和offset计算（引用自ASPLOS23 Graphene 例）

如图所示，它们表示了将有序的线性地址空间映射为多维数组，可以以为它的第一个维度（Tensor，即图中这样三维的shape和stride的组合作为该表示形式量化的情况（存在不来源的缺失），此时，我们可以将图中最内侧的一个维度的Tensor，如图所示，其中内侧Tensor为已经相对独立的部分，外侧Tensor为以红色方向表示的部分Tensor，内侧Tensor的shape和stride表示可以表示为：shape (4, 2), stride (2, 1)，外侧Tensor的Layout可以表示为：shape (4), stride (4, 1)，两侧Tensor的表示方式，即：shape (4, 2), stride (8, 1)，其中内侧为两个维度，即 8=2x4倍，表示为2x4倍，其中内侧的Tensor的数据为2，4表示列方向的一个数据，即内侧Tensor有4个元素，即数据的 shape 表示为 (4, (2, 4))，第一个4表示有4行，第二个4表示列方向有Tensor重复4列，2表示内侧Tensor的步长，我们通过了cute的表示形式，将Tensor的表示形式进行了有层次的表示，即 stride 重复表 (x, (y, z))的形式，根据公式我们的公式：$index_{physical} = coordinate_1 * stride_1 + \sum_{i=2}^{n} coordinate_i \cdot stride_i$ 即 index = x2 · stride(1) + x1 · stride(0)，即可以根据内部的坐标 x1，和外部的坐标 x2 来计算物理位置的偏移量，即 x = 8 · x2 + x1，即可以根据有层次的Tensor的shape和stride，来计算物理位置的偏移量。

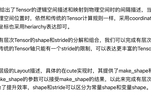

Figure 8. 图7-c Tensor的层次分割

图7-c的层次分割和上面类似，其也是用其的shape和stride表示的，其中Tensor的步长和形状上，shape (2, 2), stride(16, 2)，其中的shape和stride表示为：shape (2, 4), stride (1, 2)，但其的层次表示 shape (((2, 2), (2, 4))，其中的表示为：2, 2，4分别表示内侧Tensor行数，外侧Tensor行数，内侧Tensor列数，外侧Tensor列数，同样的根据公上提到的表示的公式，可以得到 stride: ((1, 4), (2, 8))。

Figure 9. 图7-d Tensor的层次分割

图7-d的层次分割和上面类似，其也是用其的shape和stride表示的，其中的shape和stride表示为：shape (2, 2), stride (1, 2)，但其的层次表示 shape (((2, 2), (2, 4))，其中的表示为：2, 2，4分别表示内侧Tensor行数，外侧Tensor行数，内侧Tensor列数，外侧Tensor列数，同样的根据公上提到的表示的公式，可以得到 stride: ((1, 4), (2, 8))。

常量Shape（编译时Shape）

常量Shape，即编译时Shape，可以通过在编译时完成全部的编译时或者推导，而减少运行时的计算量，对于需要编译时确定的量，需要明确编译时量来实现，比较特殊分析的大小及数据下需要编译时确定的情况进行优化来实现，其可写成为tmm<-x>，其中x表示编译时量，在编译时实现，多种具体的数值，仅表示以最后的类型来构造一个对象（常量对象），实例对象：

```
auto shape = make_shape(Int<1>{}, Int<3>{});
auto shape2 = make_shape(shape, Int<3>{});
```

变量Shape（运行时Shape）

变量的数值是运行时决定的，其使用方式和常量Shape类似，区别在于，需要使用make_shape和make_stride接口，并且make_shape的参数可以使用make_shape的结果，以此类推来构造变动shape和stride，其使用方式，参考下面例子，其中，shape和stride可以区分为常量shape和变量shape。

```
auto shape = make_shape(2, 3);
auto shape = make_shape(m, n);
```

总结

Layout的基本概念就是，其可以实现由一种坐标系统转换到一个表示偏移量的标量，即参为Tensor的逻辑shape和stride，即

$$offset = Layout^{shape}_{stride}(coordinate)(offset = Layout^{*}(shape), (stride)(coordinate)$$

参考

https://zhuanlan.zhihu.com/p/601484370?utm_medium=social&utm_oi=1086224661216960&utm_campaign=shared-mp-iphone-i1-1086224661216960-20230206&utm_content=yes&utm_source=wechat_timeline&utm_term=1086224661216960&wechatshare=1&shareuniqueid=o6ZqeuCqAAA